



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра математического обеспечения и стандартизации информационных технологий (МОСИТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ
по дисциплине «Распределенные системы управления базами данных»

Практическое занятие № 2

Студент группы *ИКБО-65-23, Олефилов Георгий Гурамович*

(подпись)

Преподаватель *Фандеев Илья Игоревич*

(подпись)

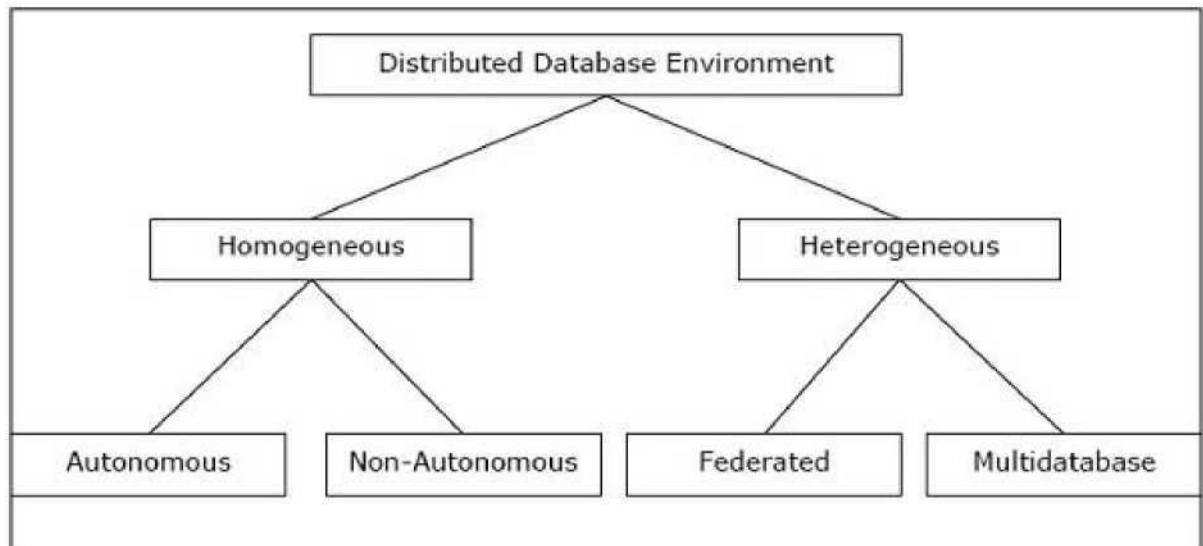
Отчет представлен «__» _____ 202__ г.

Москва 2025 г.

Теоретическое введение

Типы распределённых систем

Распределенные базы данных можно широко классифицировать на однородные и гетерогенные среды распределенных баз данных, каждая из которых имеет дополнительные подразделения, как показано на следующем рисунке:



Однородные распределенные базы данных

Разберём, как работают однородные РБД на примере сайтов (подразумевается некое веб-приложение, установленное на различных узлах). В однородной распределенной базе данных все сайты используют идентичные СУБД и операционные системы. Её свойства —

- На сайтах используется очень похожее программное обеспечение.
- Сайты используют идентичные СУБД или СУБД одного и того же производителя.
- Каждый сайт знает обо всех других сайтах и взаимодействует с другими сайтами для обработки пользовательских запросов.
- Доступ к базе данных осуществляется через единый интерфейс, как если бы это была одна база данных.

Типы однородной распределенной базы данных

Существует два типа однородной распределенной базы данных —

- **Автономный** — каждая база данных независима и функционирует самостоятельно. Они интегрированы управляющим приложением и используют передачу сообщений для обмена обновлениями данных.
- **Неавтономный** — данные распределяются по однородным узлам, а

центральная или главная СУБД координирует обновления данных по сайтам.

Гетерогенные распределенные базы данных

В гетерогенной распределенной базе данных разные сайты имеют разные операционные системы, продукты СУБД и модели данных. Его свойства —

- Различные сайты используют разные схемы и программное обеспечение.
- Система может состоять из множества СУБД, таких как реляционная, сетевая, иерархическая или объектно-ориентированная.
- Обработка запросов является сложной из-за разнородных схем.
- Обработка транзакций является сложной из-за различий в программном обеспечении.
- Сайт может не знать о других сайтах, поэтому сотрудничество при обработке пользовательских запросов ограничено.

Типы гетерогенных распределенных баз данных

- **Федеративные** — гетерогенные системы баз данных независимы по своей природе и объединены вместе, так что они функционируют как единая система баз данных.
- **Без федерации** — в системах баз данных используется центральный координационный модуль, через который осуществляется доступ к базам данных.

Практическая часть:

```
C:\Users\Giorgi>docker run --name cas1 -p 9042:9042 -e CASSANDRA_CLUSTER_NAME=OlefirovGG -e CASSANDRA_ENDPOINT_SNITCH=GossipingPropertyFileSnitch -e CASSANDRA_DC=datacenter1 -d cassandra
994c1663601b40434c47791e2f613048a6bd892db3ac6122534e7adf03528b2c
```

Рисунок 1 – Запуск контейнер с первым узлом

```
C:\Users\Giorgi>docker run --name cas2 -e CASSANDRA_SEEDS="$(docker inspect --format='{{ .NetworkSettings.IPAddress }}' cas1)" -e CASSANDRA_CLUSTER_NAME=OlefirovGG -e CASSANDRA_ENDPOINT_SNITCH=GossipingPropertyFileSnitch -e CASSANDRA_DC=datacenter1 -d cassandra
5c16403196d18dc6f7fab093d354af747774487dd235d4ba63d104e99360c813
```

Рисунок 2 – Присоединение к первому узлу следующего узла

```
C:\Users\Giorgi>docker run --name cas3 -e CASSANDRA_SEEDS="$(docker inspect --format='{{ .NetworkSettings.IPAddress }}' cas1)" -e CASSANDRA_CLUSTER_NAME=OlefirovGG -e CASSANDRA_ENDPOINT_SNITCH=GossipingPropertyFileSnitch -e CASSANDRA_DC=datacenter2 -d cassandra
6ae034779853a6f655e120ab05c703328fa26d48bc24b6fac59517f754fe8373
```

Рисунок 3 – Создание узла в «удалённом» датацентре для образования кластера

```
C:\Windows\System32>docker exec -ti cas1 nodetool status
Datacenter: datacenter1
=====
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving
-- Address      Load          Tokens   Owns (effective)  Host ID                               Rack
UN  172.17.0.2    119.83 KiB    16       61.7%             71284d50-1053-4e8a-b365-fca22afba918 rack1
UN  172.17.0.3    80.07 KiB    16       62.1%             bf04d090-6e38-4c72-9279-e05a47d7d985 rack1

Datacenter: datacenter2
=====
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving
-- Address      Load          Tokens   Owns (effective)  Host ID                               Rack
UN  172.17.0.4    114.64 KiB    16       76.2%             465d365d-991b-4370-83d3-26765f6f4b53 rack1

C:\Windows\System32>
```

Рисунок 4 – Проверка работоспособности кластера

```
C:\Users\Giorgi>docker exec -ti cas1 cqlsh
Connected to OlefirovGG at 127.0.0.1:9042
[cqlsh 6.2.0 | Cassandra 5.0.6 | CQL spec 3.4.7 | Native protocol v5]
Use HELP for help.
cqlsh>
```

Рисунок 5 - Переход в утилиту cqlsh для выполнения скрипта

```
cqlsh> CREATE KEYSPACE OlefirovGG_ks WITH replication = { 'class' : 'NetworkTopologyStrategy', 'datacenter1' : 1, 'datacenter2' : 1 };
```

Рисунок 6 – Создание пространства keyspace

```
cqlsh> USE OlefirovGG_ks;
```

Рисунок 7 – Выполнение скрипта для использования keyspace

```
cqlsh:olefirovgg_ks> CREATE TABLE OlefirovGG_ks.Student ( id int PRIMARY KEY, name text, lastname text, direction text, course int, birth_day text );
```

Рисунок 8 – Создание таблицы

```
cqlsh> CREATE KEYSPACE OlefirovGG_ks WITH replication = { 'class' : 'NetworkTopologyStrategy', 'datacenter1' : 1, 'datacenter2' : 1 };
cqlsh> USE OlefirovGG_ks;
cqlsh:olefirovgg_ks> CREATE TABLE OlefirovGG_ks.Student ( id int PRIMARY KEY, name text, lastname text, direction text, course int, birth_day text );
cqlsh:olefirovgg_ks> INSERT INTO Student (id, name, lastname, direction, course, birth_day) VALUES (1, 'Александр', 'Олефиоров', 'Информатика', 3, '2002-01-15');
cqlsh:olefirovgg_ks> INSERT INTO Student (id, name, lastname, direction, course, birth_day) VALUES (2, 'Екатерина', 'Смирнова', 'Прикладная математика', 2, '2003-03-22');
cqlsh:olefirovgg_ks> INSERT INTO Student (id, name, lastname, direction, course, birth_day) VALUES (3, 'Дмитрий', 'Иванов', 'Программная инженерия', 4, '2001-11-30');
cqlsh:olefirovgg_ks> SELECT * FROM Student;
```

id	birth_day	course	direction	lastname	name
1	2002-01-15	3	Информатика	Олефиоров	Александр
2	2003-03-22	2	Прикладная математика	Смирнова	Екатерина
3	2001-11-30	4	Программная инженерия	Иванов	Дмитрий

```
(3 rows)
cqlsh:olefirovgg_ks> exit;
```

Рисунок 9 – Заполнение ранее созданной таблицы и её вывод

```
cqlsh:olefirovgg_ks> INSERT INTO Student (id, name, lastname, direction, course, birth_day) VALUES (1, 'Александр', 'Олефиоров', 'Информатика', 3, '2002-01-15');
cqlsh:olefirovgg_ks> INSERT INTO Student (id, name, lastname, direction, course, birth_day) VALUES (2, 'Екатерина', 'Смирнова', 'Прикладная математика', 2, '2003-03-22');
cqlsh:olefirovgg_ks> INSERT INTO Student (id, name, lastname, direction, course, birth_day) VALUES (3, 'Дмитрий', 'Иванов', 'Программная инженерия', 4, '2001-11-30');
cqlsh:olefirovgg_ks> SELECT * FROM Student;
```

id	birth_day	course	direction	lastname	name
1	2002-01-15	3	Информатика	Олефиоров	Александр
2	2003-03-22	2	Прикладная математика	Смирнова	Екатерина
3	2001-11-30	4	Программная инженерия	Иванов	Дмитрий

```
(3 rows)
cqlsh:olefirovgg_ks> exit;
```

Рисунок 10 – Вывод таблицы со второго узла

Заключение:

В рамках практической работы был создан кластер Apache Cassandra, состоящий из трёх взаимосвязанных узлов. Применение контейнеризации с помощью Docker позволило быстро и удобно развернуть распределённую NoSQL-базу данных. Будучи колоночной NoSQL-системой, Cassandra отлично подходит для обработки больших объёмов информации и для работы в высоконагруженных системах. Её распределённая архитектура обеспечивает автоматическую репликацию данных между узлами, что гарантирует отказоустойчивость и поддержку горизонтального масштабирования. В итоге, развернутый и протестированный кластер, а также изученные команды Docker для управления Cassandra, предоставляют возможность создания надёжных и высокопроизводительных систем.

Контрольные вопросы:

1. Cassandra относится к распределённым **NoSQL** базам данных, а точнее — к **колоночно-ориентированным** БД. Это объясняется тем, что Cassandra:
 - Не использует реляционную модель данных (таблицы, связи и т.д.).
 - Проектирована для горизонтального масштабирования, распределённого хранения данных и высокой доступности.
 - Работает по архитектуре peer-to-peer, что позволяет обеспечить отказоустойчивость и распределённое управление данными.
2. Для загрузки (pull) образа Cassandra с Docker Hub используется команда **docker pull cassandra**